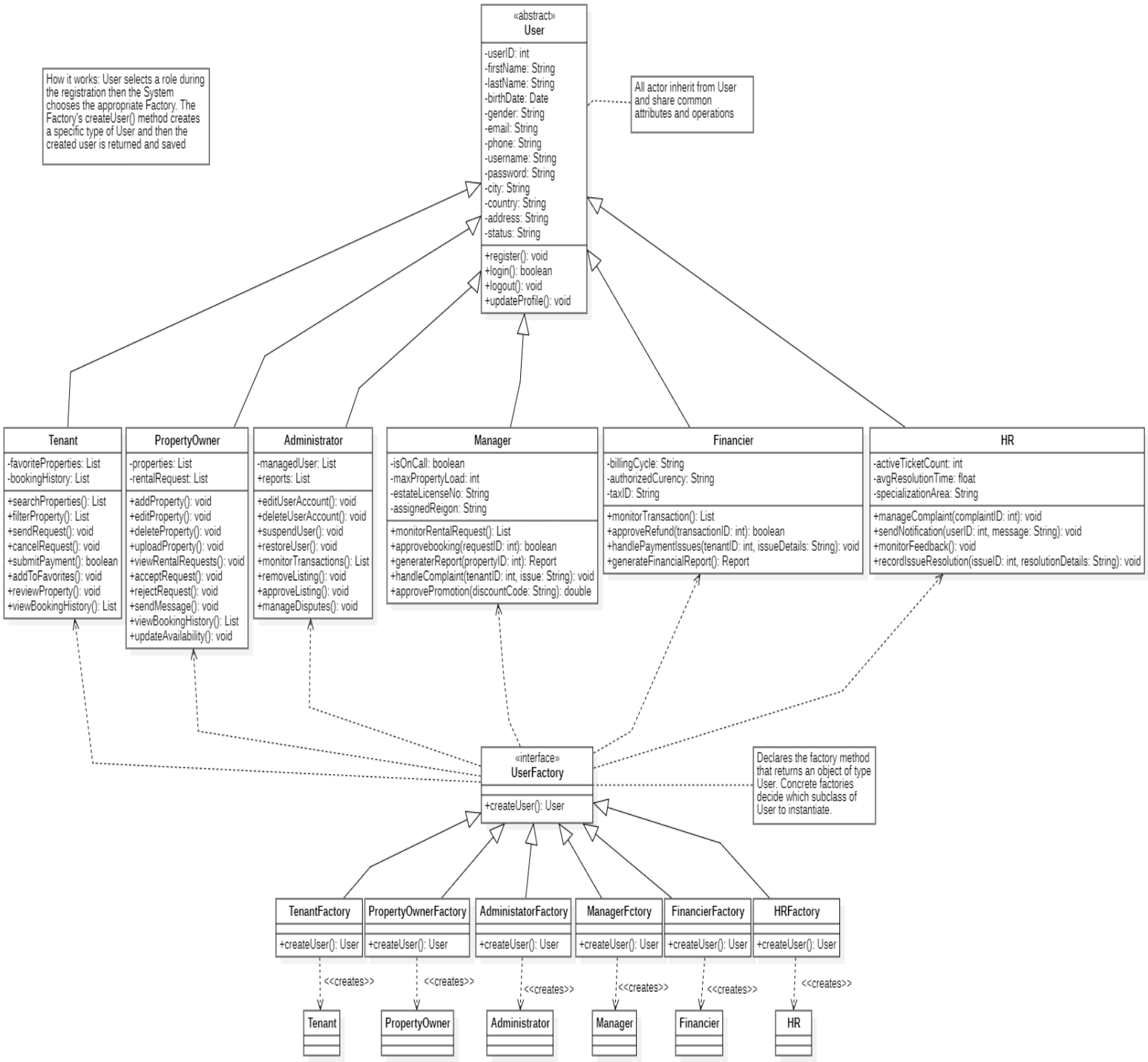
<u>Factory Method Pattern</u> (Done by Ledio Tafciu and Anisa Domi)

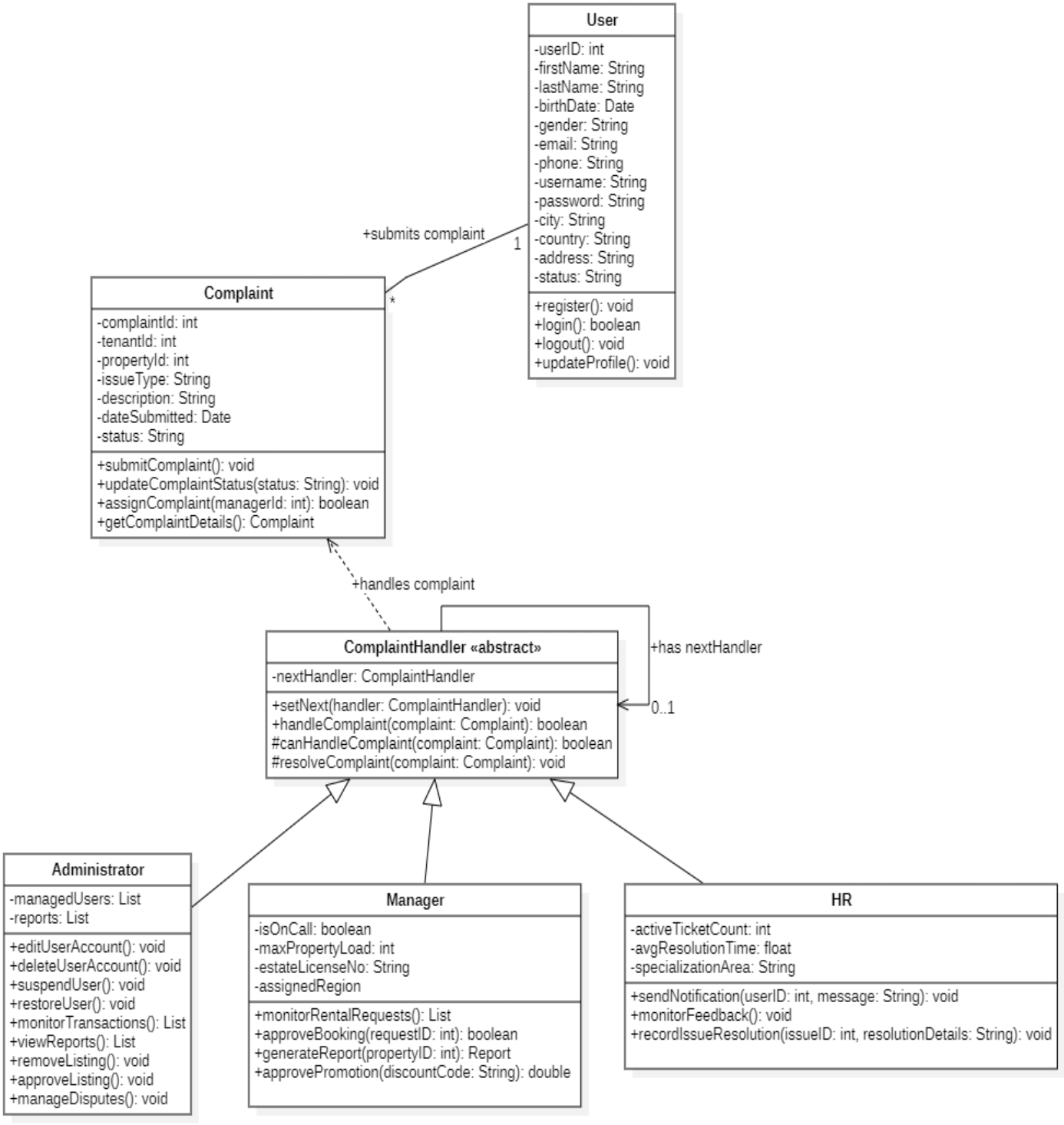
Justifying the design pattern presence➔It is justified by the functional requirements where the platform must allow all user roles to register (FR41), log in (FR42), and log out (FR43) using a unified process. When a new account is created, the system must instantiate the correct subclass of the abstract User class based on the selected role during registration. The Factory Method pattern centralizes and abstracts this object creation by delegating it to specialized factory classes such as TenantFactory, PropertyOwnerFactory, and AdministratorFactory, each responsible for producing an instance of the appropriate user type. This approach eliminates redundant registration logic for each actor and ensures all user accounts are created consistently, with the right attributes and behaviors required for their roles. As a result, the pattern provides a flexible and maintainable solution for handling user account creation and authentication as specified by FR41, FR42, and FR43.



Chain of Responsibility Pattern. (Done by Kevin Muho, Matias Bali, Ornelt Lemellari)

Justifying the design pattern presence➔ It is justified by the functional requirements where HR manages complaints(FR38), Managers review unresolved complaints(FR33), and Administrators handle disputes and system-level issues(FR26).Each handler attempts to resolve the complaint. If the complaint cannot be handled at the current level, it is forwarded to the next responsible handler in the chain.
Complaint escalation chain: HR → Manager → Administrator.
If HR cannot resolve the complaint, it forwards the complaint to the Manager.If the Manager cannot resolve it, the complaint is forwarded to the Administrator.The Administrator acts as the final handler in the chain.

HR.nextHandler = Manager Manager.nextHandler = Administrator Administrator.nextHandler =



null

Observer Pattern (Done by Haidi Toto and Desard Lybeshari)

Justifying the design pattern presence➔The *Observer Pattern* is used in the system to automatically notify users when important events occur, such as rental request updates or payment completion.

In this design:
RentalRequest and Payment act as Subjects because their status can

change.
NotificationSystem acts as the Observer, listening for these changes.
Tenant and PropertyOwner receive notifications about approvals, rejections, payments, and reminders.

For example:
-In FR17, when a property owner accepts or rejects a rental request, the tenant is notified.
-In FR8 and FR39, payment updates and reminders are also sent automatically.